UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Centro Universitario de Puerto Barrios, Izabal
Facultad de Ingeniería en Sistemas

DOCUMENTACIÓN TÉCNICA INTEGRAL

Meal Planner

Aplicación móvil para la planificación de alimentación saludable

Análisis, requerimientos, arquitectura, diseño
y estado de los avances documentales

Presentado por
José Rodríguez
Carné 0909-23-5430

Análisis de Sistemas II
Docente: Ing. Pablo Menjívar

Puerto Barrios, Izabal
24 de octubre de 2026

Resumen

Meal Planner es un proyecto de aplicación móvil orientado a organizar comidas diarias y semanales, registrar alimentos consumidos, consultar el historial y conservar recetas favoritas. El sistema contempla el registro de usuarios, el inicio y cierre de sesión, la administración del perfil y la definición de objetivos alimenticios. Los avances describen un cliente construido con React Native, Expo y TypeScript, conectado mediante una API REST a un backend centralizado en Node.js (Rodríguez, s. f.-a, pp. 1-2; s. f.-b, pp. 7-10).

Este informe adopta la estructura de la documentación técnica proporcionada como referencia y la adapta al dominio de Meal Planner. Integra el planteamiento del proyecto, los requerimientos funcionales y no funcionales, las vistas de arquitectura, el modelo de datos, los casos de uso y las secuencias de interacción. También presenta criterios de validación y una ruta de trabajo para completar el desarrollo.

La arquitectura se caracteriza como cliente-servidor con un backend monolítico organizado por módulos. La separación entre autenticación, planificación y recetas expresa responsabilidades internas; no demuestra servicios desplegados de manera independiente. Por ello, el proyecto no se clasifica como una arquitectura de microservicios.

El estado descrito se basa en los dos documentos de avance. La vista física indica que todavía no se utiliza una base de datos, mientras que el tercer entregable presenta un modelo entidad-relación para una fase posterior. En consecuencia, la persistencia y los flujos que la requieren se documentan como diseño propuesto. No se aportaron código fuente de la aplicación, capturas de ejecución ni resultados de pruebas que permitan certificar funciones implementadas, rendimiento o compatibilidad móvil.

Palabras clave: planificación de comidas, aplicación móvil, React Native, API REST, cliente-servidor, arquitectura modular.

Criterio de lectura

Término utilizado	Significado en este documento
Documentado	La función o decisión aparece en los avances entregados.
Propuesto	Detalle de diseño desarrollado para completar o aclarar el análisis.
No verificado	No se dispone de evidencia de ejecución o medición para confirmar su funcionamiento.

Contenido

Introducción y planteamiento del problema	4
Justificación, objetivos y alcance	5
Método de integración y estado documental	6
Requerimientos funcionales	7
Requerimientos no funcionales	8
Arquitectura cliente-servidor y vista física	9
Vista lógica y organización de desarrollo	10
Vista de procesos y tecnologías	11
Diseño de datos: modelo entidad-relación	12
Diccionario de datos propuesto	13
Reglas de negocio e integridad propuestas	14
Casos de uso y trazabilidad	15
Especificación de casos de uso: acceso y perfil	16
Especificación de casos de uso: alimentación	17
Secuencia de autenticación	18
Secuencias de planificación y consumo	19
Funciones previstas por pantalla y contrato de API	20
Seguridad, privacidad y límites operativos	21
Verificación: evidencia y plan de pruebas	22
Brechas y plan de trabajo recomendado	23
Guía de operación prevista y preparación técnica	24
Conclusiones	25

Introducción y planteamiento del problema

Meal Planner propone reunir en una aplicación móvil las actividades de planificación de comidas y registro de alimentación personal. Su propósito es que el usuario pueda organizar lo que piensa consumir, registrar lo que efectivamente consumió y consultar esa información desde un mismo entorno. El alcance funcional procede del segundo entregable del proyecto y la interpretación arquitectónica se complementa con el análisis posterior de MealPlanner (Rodríguez, s. f.-b, pp. 7-12; s. f.-a, pp. 1-2).

Antecedentes

Los avances identifican los módulos de autenticación, gestión de usuarios, perfil, planificador, registro de alimentos, recetas favoritas y comunicación con la API. También incluyen vistas física y de desarrollo, un diagrama de casos de uso, una secuencia de inicio de sesión y un modelo entidad-relación. Estos elementos establecen una base de análisis y diseño para continuar el proyecto.

Las primeras seis páginas del archivo de ideas pertenecen a una propuesta de gestión de clientes con C#, .NET y SQL Server. Su tema, tecnologías y alcance difieren de Meal Planner. Por coherencia, no se utilizan como requerimientos ni como evidencia técnica de este sistema.

Planteamiento del problema

El problema de diseño consiste en ofrecer un mecanismo organizado para relacionar el perfil del usuario, sus objetivos, su planificación y sus registros diarios de alimentación. Sin una estructura común, el plan de comidas y el consumo real pueden quedar separados, lo que dificulta revisar antecedentes y mantener continuidad en el seguimiento personal.

Meal Planner busca resolver esa necesidad mediante un cliente móvil y una API que concentre las operaciones. La aplicación debe permitir consultar y actualizar información propia, conservar relaciones consistentes entre planes y comidas, y mostrar los resultados de cada operación de manera comprensible.

Este planteamiento se formula a partir de las funciones solicitadas en los avances. Los documentos no incluyen entrevistas, encuestas ni mediciones sobre hábitos alimenticios, tiempos de uso o errores de los usuarios. Por tanto, la necesidad se presenta como fundamento del proyecto académico y no como una conclusión validada mediante un estudio de campo.

Pregunta orientadora

> ¿Cómo organizar la planificación y el registro de alimentación de una persona mediante una aplicación móvil con una API centralizada, manteniendo claridad de uso, separación de responsabilidades y consistencia de los datos?

Justificación, objetivos y alcance

Justificación

El proyecto aporta una solución de organización personal al integrar en un mismo flujo el perfil, los objetivos alimenticios, los planes y el consumo registrado. La distinción entre lo planificado y lo consumido permite conservar un historial comprensible sin asumir que todas las comidas programadas fueron realizadas.

Desde la perspectiva académica, permite aplicar análisis de requerimientos, modelado de datos, diseño de interfaces móviles y comunicación cliente-servidor. La organización modular favorece distribuir responsabilidades y preparar la evolución del sistema sin exigir desde esta fase servicios independientes.

Objetivo general

Desarrollar una aplicación móvil de planificación de alimentación que permita administrar el perfil y los objetivos del usuario, organizar comidas diarias y semanales, registrar alimentos consumidos y consultar su historial, mediante un cliente React Native con Expo y TypeScript conectado a una API REST en Node.js.

Objetivos específicos

- Definir el registro, la autenticación y el cierre de sesión de los usuarios.
- Administrar el perfil y los objetivos de bajar de peso, aumentar masa muscular o mantener peso.
- Organizar comidas por día y semana, con opciones de edición y eliminación.
- Registrar el consumo de alimentos y consultar los registros anteriores.
- Permitir guardar recetas favoritas y relacionarlas con el usuario.
- Establecer una arquitectura modular y criterios de validación para la comunicación, la seguridad y el rendimiento.

Alcance y delimitaciones

Se incluyen los diez requerimientos funcionales y ocho no funcionales del segundo entregable. La persistencia se mantiene como diseño pendiente de materialización. No se incorporan como compromisos confirmados la compra de alimentos, pagos, inventarios, integración con dispositivos de salud, recomendaciones automáticas, publicación en tiendas o funcionamiento sin conexión. Tales funciones requieren una decisión de alcance adicional.

Método de integración y estado documental

Se revisaron los tres PDF proporcionados, se extrajo su contenido textual y se inspeccionaron visualmente los diagramas incluidos como imágenes. El primer PDF se utilizó como referencia de estructura académica y profundidad técnica. Los dos restantes se utilizaron como fuentes del contenido de Meal Planner. No se realizó una auditoría de software ni una validación de una aplicación ejecutable.

Material	Contenido utilizado	Tratamiento
Documentación.pdf 25 páginas	Estructura: resumen, problema, objetivos, alcance, requisitos, arquitectura, datos, casos de uso, secuencias, verificación y conclusiones.	Se adapta la organización. No se transfieren hechos ni resultados del sistema de facturación.
MealPlanner.pdf 2 páginas	React Native, Expo, TypeScript, Node.js; API REST; cliente-servidor y ausencia de microservicios.	Se integra como aclaración arquitectónica posterior.
Idea de proyecto analisis (4).pdf 12 páginas	Pp. 7-8: requisitos y módulos. Pp. 9-10: vistas física y de desarrollo. Pp. 11-12: casos de uso, secuencia y entidades.	Se integran los avances de Meal Planner. Pp. 1-6 se excluyen por corresponder a otro proyecto.

Conciliación de diferencias entre los avances

La vista física indica que aún no se utiliza una base de datos. La secuencia de inicio de sesión incluye una consulta a base de datos y el modelo entidad-relación se presenta como propuesta futura. Para mantener consistencia, esos diagramas se interpretan como el comportamiento objetivo cuando exista persistencia; no como prueba de que una base ya esté instalada.

La API forma parte del sistema, junto con la aplicación móvil. En el diagrama de casos de uso de este informe se mantiene al usuario como actor externo y se ubica al backend dentro del límite del sistema. Esto aclara la interacción sin alterar las funciones del avance.

Nota. Los detalles añadidos sobre validación, autorización, contratos de API y pruebas se identifican como propuestas. No se atribuyen resultados de ejecución al proyecto.

Requerimientos funcionales

Se conservan los identificadores RF-01 a RF-10 del segundo entregable. Todos están documentados y su implementación permanece sin verificar (Rodríguez, s. f.-b, p. 7). Los criterios siguientes precisan el comportamiento esperado para una futura aceptación.

ID	Requerimiento del avance	Criterio de aceptación propuesto
RF-01	Registrar nuevos usuarios.	Una cuenta válida puede crearse; correo duplicado o datos obligatorios ausentes producen un aviso y no generan otra cuenta.
RF-02	Iniciar y cerrar sesión.	Credenciales válidas habilitan la sesión; las inválidas no conceden acceso. Al cerrar, se elimina el acceso activo desde el cliente.
RF-03	Crear y editar el perfil personal.	El usuario consulta y modifica su propio perfil; los cambios válidos se recuperan al consultarlo de nuevo con persistencia habilitada.
RF-04	Establecer objetivos alimenticios.	Se puede seleccionar bajar de peso, aumentar masa muscular o mantener peso y consultar el objetivo elegido.
RF-05	Planificar comidas diarias y semanales.	Se crea un plan para un intervalo válido y se agregan comidas asociadas a días incluidos en ese intervalo.
RF-06	Registrar alimentos consumidos durante el día.	Se registra alimento, fecha, cantidad y unidad; el registro queda asociado al usuario autenticado.
RF-07	Consultar el historial de comidas registradas.	Se recuperan los registros propios y se presenta una vista comprensible cuando existen datos o cuando el historial está vacío.
RF-08	Guardar recetas favoritas.	Una receta guardada queda asociada al usuario y puede recuperarse en su listado de favoritas.
RF-09	Editar o eliminar registros de comidas.	Un registro propio puede modificarse o eliminarse tras validar la operación; no puede alterarse un registro ajeno.
RF-10	Consumir una API REST desarrollada en Node.js.	El cliente intercambia solicitudes y respuestas con el backend, gestiona errores y actualiza la pantalla según la respuesta.

Nota. RF-09 no distingue de manera expresa entre comida planificada y consumo. Se propone cubrir ambas operaciones y confirmar esa interpretación al cerrar el alcance.

Requerimientos no funcionales

Los RNF mantienen su numeración original. La columna de comprobación define evidencia por obtener; no representa una validación ya realizada (Rodríguez, s. f.-b, pp. 7-8).

ID	Requerimiento documentado	Comprobación propuesta
RNF-01	Compatibilidad con Android e iOS mediante React Native y Expo.	Ejecutar flujos críticos en ambas plataformas y registrar versiones de sistema, dispositivos y resultado.
RNF-02	Interfaz intuitiva y fácil de utilizar.	Observar tareas de registro, planificación y consumo con usuarios; registrar dificultades y tasa de finalización.
RNF-03	Respuesta no mayor de 3 segundos en operaciones normales.	Definir operaciones, volumen, concurrencia y red. Medir desde la acción del usuario hasta la presentación del resultado; reportar las condiciones.
RNF-04	Comunicación HTTP mediante APIs REST.	Verificar métodos, recursos, estados y formato de respuestas. Para datos de sesión se propone HTTPS en ambientes compartidos o publicados.
RNF-05	Uso de TypeScript para mantenibilidad.	Revisar tipado de modelos y contratos, compilación y manejo de respuestas. El alcance exacto del tipado del backend debe confirmarse.
RNF-06	Backend desarrollado con Node.js.	Verificar el entorno de ejecución y la implementación del servidor cuando se entregue el código.
RNF-07	Crecimiento mediante arquitectura modular.	Revisar separación de presentación, servicios, reglas y acceso a datos; comprobar que un cambio de módulo no exige duplicar lógica.
RNF-08	Manejo seguro de la información del usuario durante la sesión.	Evaluar autenticación, autorización por propietario, manejo de credenciales, transporte y cierre o expiración de sesión.

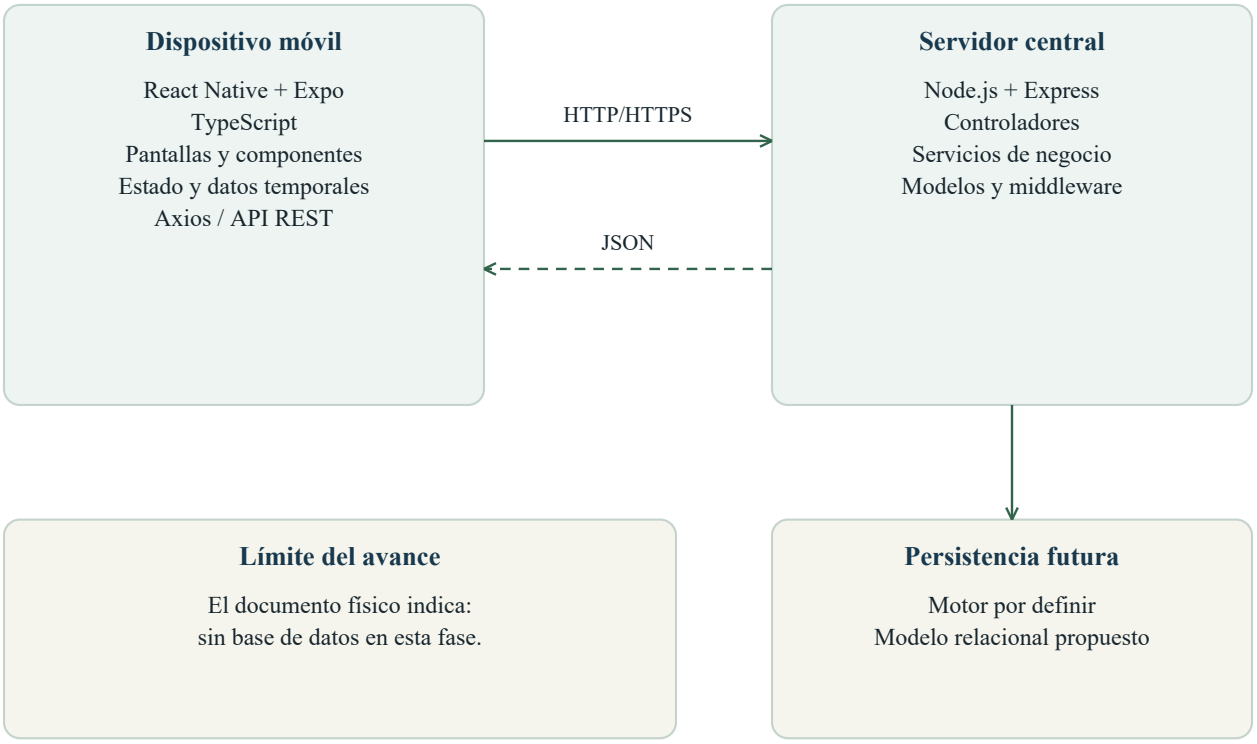
Condiciones pendientes de definición

El umbral de tres segundos carece de una carga y red de referencia. Tampoco se especifican versiones mínimas de Android e iOS, tamaño esperado del catálogo, política de sesión ni criterios cuantitativos de usabilidad. Estos acuerdos deben fijarse antes de afirmar cumplimiento.

Arquitectura cliente-servidor y vista física

Meal Planner se describe como un sistema cliente-servidor: la aplicación móvil envía solicitudes a una API REST en Node.js y presenta las respuestas al usuario. El análisis arquitectónico establece un único backend centralizado y no aporta evidencia de servicios desplegados de forma independiente (Rodríguez, s. f.-a, pp. 1-2).

Figura 1. *Distribución física y límite de persistencia*



Nota. Redibujo a partir de la vista física de Rodríguez (s. f.-b, p. 9). La conexión con persistencia representa una extensión futura; no una base de datos existente.

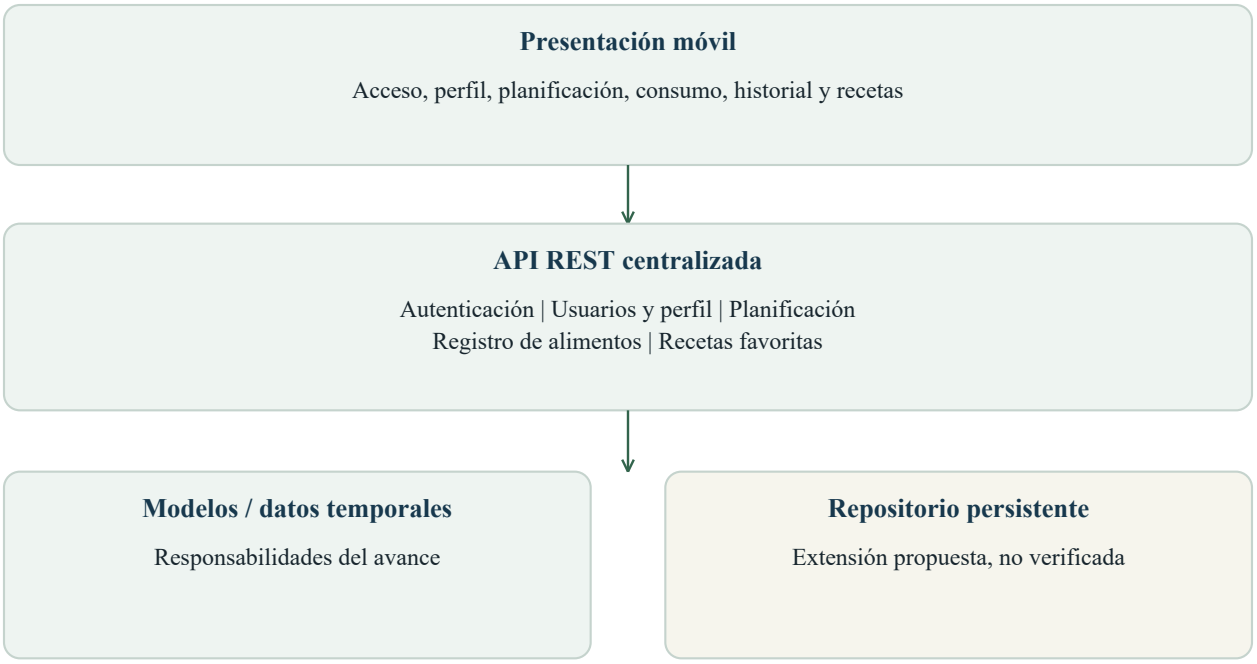
Responsabilidad de cada extremo

El cliente gestiona pantallas, formularios, navegación y presentación de resultados. Puede validar campos para orientar al usuario, pero el servidor debe repetir las reglas que protegen la operación. El backend recibe las peticiones, coordina autenticación, planificación y registros, y devuelve respuestas de la API.

La red conecta ambos componentes. Los avances muestran HTTP/HTTPS y respuestas JSON; para un entorno con información real se propone proteger el transporte con HTTPS. No se especifican proveedor de alojamiento, dominio, puerto ni ambiente publicado.

Vista lógica y organización de desarrollo

Figura 2. *Capas y módulos de responsabilidad*



Nota. Síntesis de los módulos y de la organización representada en los avances. La organización por capas es interna a una API única.

Módulo o carpeta	Responsabilidad documentada o propuesta
app/	Pantallas y navegación. El avance muestra rutas de autenticación y pestañas de inicio, planificador, recetas y perfil.
components/ y assets/	Elementos de interfaz reutilizables y recursos visuales.
services/	Comunicación HTTP con la API; la vista de desarrollo muestra Axios.
hooks/ y context/	Lógica reutilizable y estado compartido, incluido el contexto de autenticación.
types/ y utils/	Definiciones TypeScript y funciones auxiliares comunes.
Backend: controladores, servicios, modelos y middleware	Atención de peticiones, reglas del dominio, estructura de datos, autenticación, validación y errores.

Nota. La estructura de carpetas procede del diagrama de desarrollo (Rodríguez, s. f.-b, p. 10). No se inspeccionó un repositorio que confirme nombres o contenido real de archivos.

Vista de procesos y tecnologías

Proceso principal documentado

1. El usuario abre la aplicación y solicita iniciar sesión.
2. La aplicación envía las credenciales a la API.
3. El servidor valida la solicitud y responde con el resultado de autenticación.
4. El usuario accede al menú y selecciona el planificador.
5. La aplicación permite registrar o consultar comidas y envía la operación al servidor.
6. El backend procesa la solicitud y devuelve una respuesta.
7. El cliente actualiza la interfaz con los datos confirmados o muestra un error.

Tecnologías identificadas en los avances

Tecnología	Papel y alcance de la evidencia
React Native + Expo	Desarrollo de la aplicación móvil con Android e iOS como plataformas objetivo; ejecución en ambas pendiente de verificar.
TypeScript	Lenguaje indicado para el proyecto móvil y mostrado también en la vista del backend; no se revisó una compilación.
Node.js + Express	Backend central y API REST. Express se identifica en la vista física.
Axios y JSON	Cliente HTTP y formato de intercambio señalados en los diagramas.
Expo Router	Navegación indicada en la vista de desarrollo; no se comprobaron rutas en ejecución.
AsyncStorage	Almacenamiento local de datos temporales en la vista física; no equivale a la persistencia central propuesta.
Base de datos	No utilizada en la fase descrita por la vista física. Motor, migraciones y operación siguen por definir.

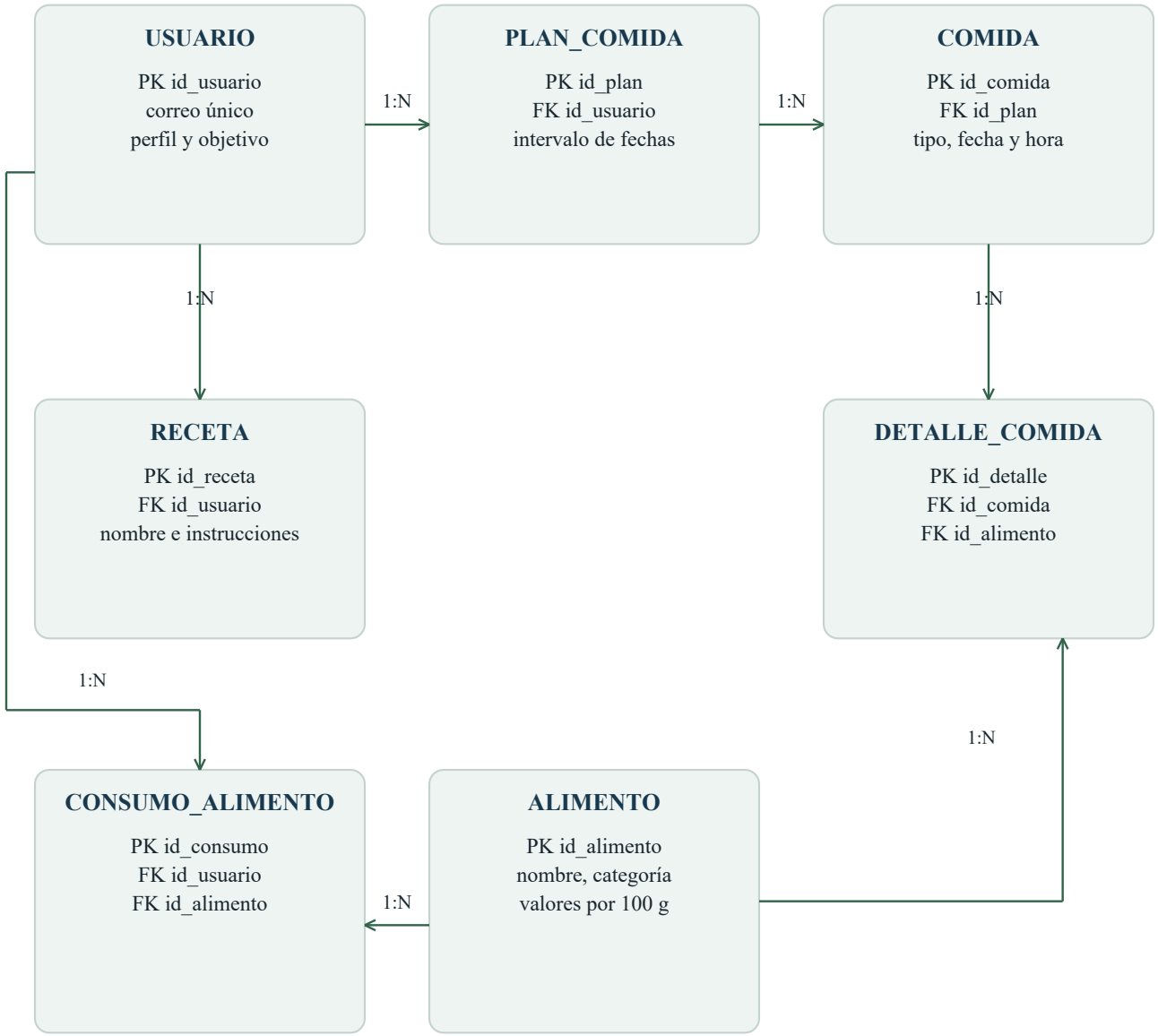
Escalabilidad y despliegue

La evolución inicial puede conservar una sola API organizada por módulos. El paso a microservicios requeriría justificar una necesidad concreta y definir despliegues, comunicación y operación independientes. Los avances no exigen ese cambio ni documentan una infraestructura de producción. Antes de desplegar, corresponde resolver persistencia, configuración, secretos, transporte y pruebas de plataforma.

Diseño de datos: modelo entidad-relación

El tercer entregable presenta siete entidades: USUARIO, PLAN_COMIDA, COMIDA, DETALLE_COMIDA, ALIMENTO, CONSUMO_ALIMENTO y RECETA. Se conserva su estructura conceptual como referencia para la futura persistencia (Rodríguez, s. f.-b, p. 12).

Figura 3. *Modelo conceptual propuesto para persistencia*



Nota. PK: clave primaria. FK: clave foránea. Las relaciones se leen desde la entidad de origen hacia el detalle con cardinalidad 1:N. El diagrama es una propuesta, no un esquema desplegado.

Interpretación de relaciones

Un usuario tiene varios planes, recetas y consumos. Cada plan contiene varias comidas; una comida contiene varios detalles y cada detalle referencia un alimento. Un alimento puede aparecer en varios detalles y consumos. DETALLE_COMIDA resuelve la relación entre comidas y alimentos, conservando cantidad y unidad.

Diccionario de datos propuesto

Entidad	Campos del modelo de origen	Interpretación y precisión de diseño
USUARIO	id_usuario (PK), nombre, apellido, correo, contraseña, fecha_registro, objetivo, peso_actual, estatura.	Identidad y perfil. Correo único. Se propone persistir un hash de contraseña, nunca el texto original. Unidades de peso y estatura por acordar.
PLAN_COMIDA	id_plan (PK), id_usuario (FK), nombre_plan, fecha_inicio, fecha_fin, observaciones.	Plan perteneciente a un usuario. El intervalo habilita organización diaria o semanal.
COMIDA	id_comida (PK), id_plan (FK), tipo_comida, fecha, hora.	Comida planificada; el avance ejemplifica desayuno, almuerzo, cena y snack.
DETALLE_COMIDA	id_detalle (PK), id_comida (FK), id_alimento (FK), cantidad, unidad_medida.	Asocia un alimento y su cantidad con una comida planificada.
ALIMENTO	id_alimento (PK), nombre, categoria, calorias_por_100g, proteinas, carbohidratos, grasas.	Catálogo alimentario. Se debe definir origen, unidades y criterio de validación de los datos nutricionales.
CONSUMO_ALIMENTO	id_consumo (PK), id_usuario (FK), id_alimento (FK), fecha, cantidad, unidad_medida, calorias_totales.	Consumo real, independiente de la planificación. El cálculo de calorías requiere equivalencia de unidad y fuente definida.
RECETA	id_receta (PK), id_usuario (FK), nombre, descripcion, instrucciones, imagen_url, fecha_creacion.	Receta personal guardada como favorita. El modelo no define ingredientes de receta ni catálogo compartido.

Nota. Los campos proceden del diagrama de origen. Tipos físicos, longitudes, obligatoriedad, índices y políticas de eliminación deben definirse durante el diseño de la base.

Decisiones abiertas del modelo

Falta definir si un consumo puede vincularse con la comida planificada que lo originó; el esquema actual no contiene esa relación. También se requiere acordar zonas horarias e historial de objetivos. RECETA corresponde a registros personales: un catálogo compartido requeriría una relación adicional de favoritos. Estas decisiones deben resolverse antes de las migraciones.

Reglas de negocio e integridad propuestas

Las siguientes reglas desarrollan los requerimientos y las relaciones del modelo. Son criterios de diseño para implementar y probar; no reglas verificadas en el código.

Regla	Criterio propuesto
R-01. Identidad	Normalizar y validar el correo; impedir cuentas duplicadas. El identificador del usuario debe obtenerse de la sesión validada.
R-02. Propiedad	Un usuario solo puede consultar y modificar su perfil, planes, comidas, consumos y recetas. El servidor comprueba esta relación.
R-03. Objetivo	Aceptar las opciones establecidas en RF-04. Elegir un objetivo no genera por sí mismo una prescripción alimenticia.
R-04. Intervalos	La fecha final de un plan no puede ser anterior a su inicio; cada comida debe pertenecer al intervalo del plan.
R-05. Cantidades	Las cantidades de alimentos deben ser positivas y acompañarse de una unidad reconocida.
R-06. Catálogo	Cada detalle y consumo debe referenciar un alimento existente. No aceptar identificadores inexistentes o desvinculados.
R-07. Consumo real	Crear un plan no registra automáticamente consumo. El usuario confirma el alimento consumido mediante una operación independiente.
R-08. Operaciones completas	Guardar un plan con sus comidas o un cambio con sus detalles de forma coherente, evitando registros parciales ante errores.
R-09. Eliminación	Confirmar acciones de eliminación y definir tratamiento de dependencias; evitar detalles huérfanos al eliminar una comida.
R-10. Cálculo opcional	Solo calcular calorías si existe una equivalencia válida entre la cantidad y la referencia nutricional. No asumir que una porción equivale a gramos.

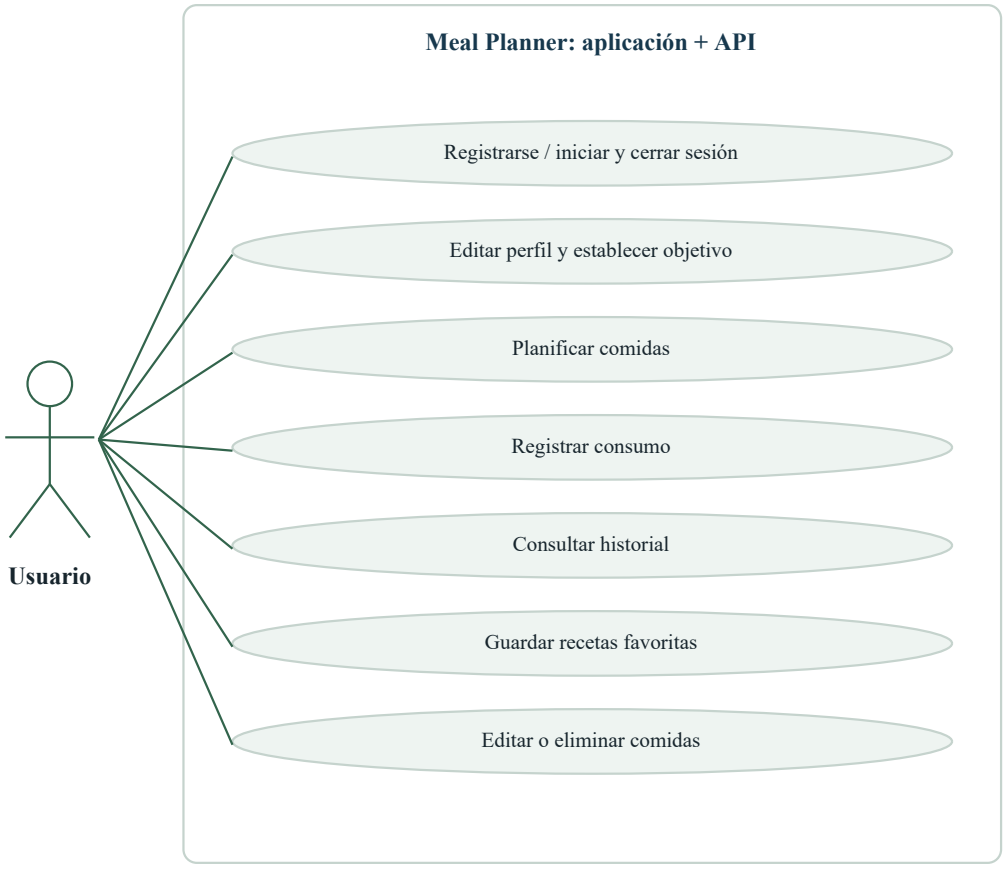
Unidades y datos derivados

Si se adopta el dato calorias_por_100g, una cantidad expresada en gramos podría calcularse como cantidad_g × calorias_por_100g / 100. Esa fórmula no resuelve unidades como taza o porción sin una conversión definida. Su uso es una propuesta técnica asociada al modelo y no una función confirmada por los requerimientos.

Casos de uso y trazabilidad

El actor principal es el usuario de la aplicación. El límite comprende cliente y servidor; la API es un componente interno. Se mantienen las capacidades del diagrama entregado y se hacen explícitos cierre de sesión, objetivos y edición o eliminación.

Figura 4. *Interacciones del usuario con Meal Planner*



Nota. Redibujo y normalización del diagrama de Rodríguez (s. f.-b, p. 11). Las relaciones con la API se explican en secuencias, sin representarla como una persona usuaria externa.

Casos de uso	Requisitos relacionados
CU-01 Registro; CU-02 Inicio de sesión; CU-03 Cierre de sesión	RF-01, RF-02, RF-10
CU-04 Perfil; CU-05 Objetivo alimenticio	RF-03, RF-04, RF-10
CU-06 Planificación; CU-07 Consumo; CU-08 Historial	RF-05, RF-06, RF-07, RF-10
CU-09 Recetas favoritas; CU-10 Editar o eliminar comidas	RF-08, RF-09, RF-10

Especificación de casos de uso: acceso y perfil

CU-01. Registrar usuario

Actor: Usuario. **Precondición:** No requiere sesión activa.
Flujo principal: Abre el registro, ingresa los datos solicitados y envía el formulario. La API valida la cuenta y confirma su creación.
Alternativas: Correo inválido o repetido, campos incompletos o falta de conexión: informar el motivo y permitir corregir o reintentar.
Postcondición: Cuenta disponible para autenticarse cuando exista almacenamiento persistente.

CU-02. Iniciar sesión

Actor: Usuario. **Precondición:** Existe una cuenta registrada.
Flujo principal: Ingresa correo y contraseña. La aplicación solicita autenticación a la API; si es aceptada, conserva la información mínima de sesión y abre la pantalla principal.
Alternativas: Credenciales incorrectas, sesión no disponible o error de red: denegar acceso y mostrar un aviso comprensible.
Postcondición: Sesión activa según la política que se implemente.

CU-03. Cerrar sesión

Actor: Usuario. **Precondición:** Hay una sesión activa en el cliente.
Flujo principal: Selecciona cerrar sesión. La aplicación limpia el estado y la credencial local; se solicita invalidación al servidor si el mecanismo elegido la admite.
Alternativas: Si no hay conexión, al menos debe bloquearse el acceso local; el vencimiento o la revocación remota dependen del diseño de sesión.
Postcondición: La aplicación vuelve al acceso y no muestra datos de la sesión anterior.

CU-04. Crear o editar perfil

Actor: Usuario. **Precondición:** Usuario autenticado.
Flujo principal: Abre Perfil, consulta datos, modifica los campos admitidos y guarda. El servidor valida identidad y contenido, y devuelve el perfil actualizado.
Alternativas: Campos inválidos o intento de modificar otro perfil: rechazar y conservar la versión válida.
Postcondición: Perfil propio actualizado; recuperación posterior por comprobar con persistencia.

CU-05. Establecer objetivo alimenticio

Actor: Usuario. **Precondición:** Usuario autenticado.
Flujo principal: En su perfil elige bajar de peso, aumentar masa muscular o mantener peso y guarda la selección.
Alternativas: Valor fuera de las opciones permitidas o fallo de guardado: mostrar aviso sin confirmar el cambio.
Postcondición: Objetivo asociado al perfil, sin generar automáticamente un plan nutricional.

Nota. Especificaciones elaboradas a partir de RF-01 a RF-04. Detalles de validación y manejo de sesión se proponen para cerrar el diseño.

Especificación de casos de uso: alimentación

CU-06. Planificar comidas

Actor: Usuario. **Precondición:** Usuario autenticado; alimentos disponibles si se agregan detalles.
Flujo principal: Selecciona día o semana, crea un plan y agrega comidas con tipo, fecha, hora y alimentos. Envía el plan a la API y revisa el resultado.
Alternativas: Intervalo inválido, fecha fuera del plan o alimento inexistente: impedir confirmación hasta corregir.
Postcondición: Plan propio disponible para consulta; no se crea consumo real automáticamente.

CU-07. Registrar consumo de alimentos

Actor: Usuario. **Precondición:** Usuario autenticado y alimento identificado.
Flujo principal: Selecciona alimento, fecha, cantidad y unidad. Confirma el registro y la aplicación actualiza la información diaria.
Alternativas: Cantidad no positiva, unidad desconocida o error de comunicación: informar y permitir corregir sin duplicar el registro.
Postcondición: Consumo asociado al usuario y disponible en su historial.

CU-08. Consultar historial

Actor: Usuario. **Precondición:** Usuario autenticado.
Flujo principal: Abre el historial y solicita sus registros. El servidor devuelve los consumos propios y la aplicación los organiza para revisión.
Alternativas: Sin registros: mostrar estado vacío. Sin conexión o sesión vencida: informar y ofrecer el siguiente paso apropiado.
Postcondición: Historial consultado sin alterar los datos; filtros por fecha se proponen, no están descritos como una función comprobada.

CU-09. Guardar receta favorita

Actor: Usuario. **Precondición:** Usuario autenticado y receta disponible o datos para crearla.
Flujo principal: Selecciona guardar la receta. La API la asocia a su cuenta y confirma su presencia en el listado.
Alternativas: Datos incompletos o guardado repetido: validar y evitar duplicados según el criterio acordado.
Postcondición: Receta disponible entre las favoritas del usuario. El origen del catálogo debe definirse.

CU-10. Editar o eliminar una comida

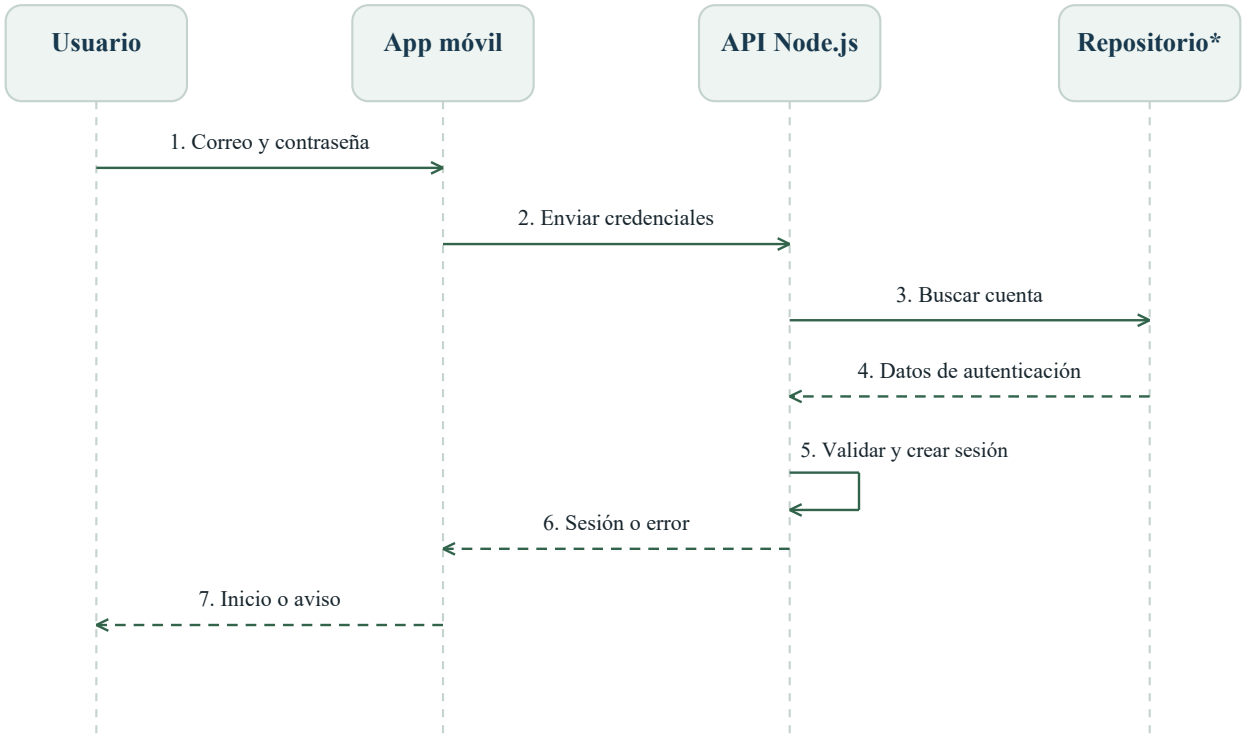
Actor: Usuario. **Precondición:** Existe un registro propio.
Flujo principal: Selecciona una comida o consumo, modifica sus datos o solicita eliminarlo. La API valida propiedad e integridad y confirma el resultado.
Alternativas: Registro ajeno, inexistente o error de validación: rechazar. Para eliminar, solicitar confirmación previa.
Postcondición: Consulta e historial reflejan el resultado confirmado; la cobertura de ambos tipos de registro se propone para RF-09.

Nota. Los flujos concretan RF-05 a RF-09. Su operación depende de implementación y validación posteriores.

Secuencia de autenticación

La secuencia integra el inicio de sesión del tercer entregable. En el avance aparece una base de datos como participante, aunque la vista física declara que todavía no se utiliza. Aquí se presenta como repositorio futuro y se distingue la respuesta exitosa del rechazo por credenciales inválidas.

Figura 5. *Inicio de sesión: comportamiento objetivo*



*Nota. * Repositorio de usuarios propuesto para la fase de persistencia. La emisión de un token aparece en el diagrama original, pero no se especifican su formato, duración o almacenamiento seguro (Rodríguez, s. f.-b, p. 11).*

Validaciones y alternativas propuestas

La API valida la estructura de la solicitud, localiza la cuenta y verifica la contraseña mediante un mecanismo de hash. Si las credenciales son válidas, crea la sesión definida para el proyecto. La aplicación no debe interpretar una respuesta incompleta como un acceso autorizado.

Ante credenciales inválidas, se propone una respuesta genérica que no revele si el correo está registrado. Si el servidor no responde, el cliente muestra un error de comunicación y permite reintentar. El mecanismo concreto de autenticación y revocación debe elegirse antes de implementar la secuencia definitiva.

Estado: secuencia documentada como diseño. No se verificaron autenticación real, expiración de sesión, almacenamiento de credenciales ni consulta a una base de datos.

Secuencias de planificación y consumo

Las dos operaciones se mantienen separadas: la planificación registra una intención y el consumo registra un hecho. Los diagramas siguientes amplían RF-05 y RF-06 para orientar la implementación.

Figura 6. *Guardar un plan de comidas*

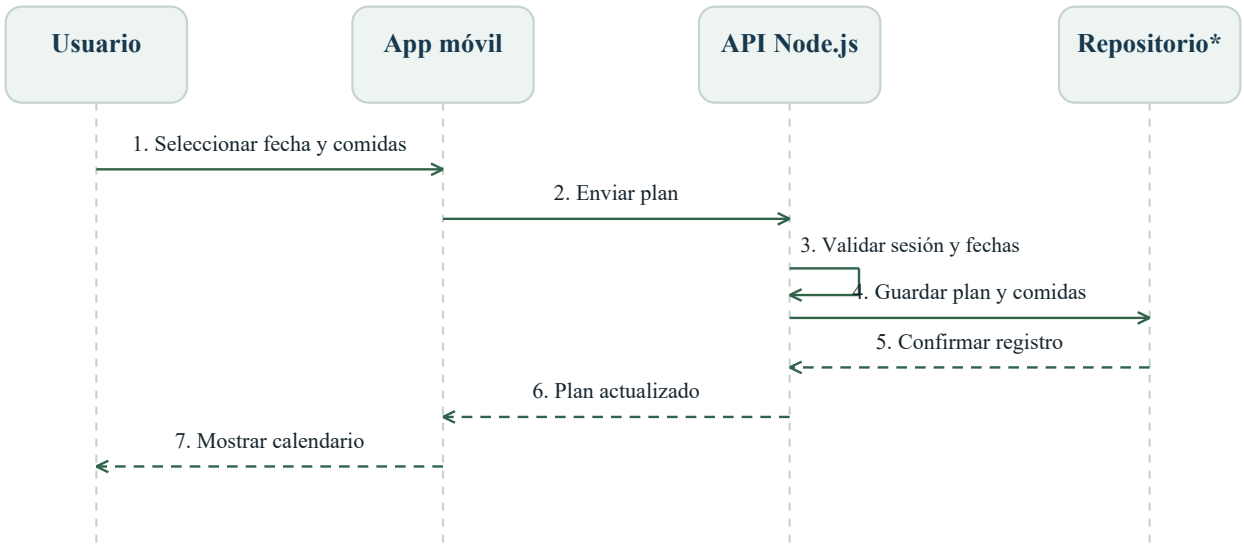
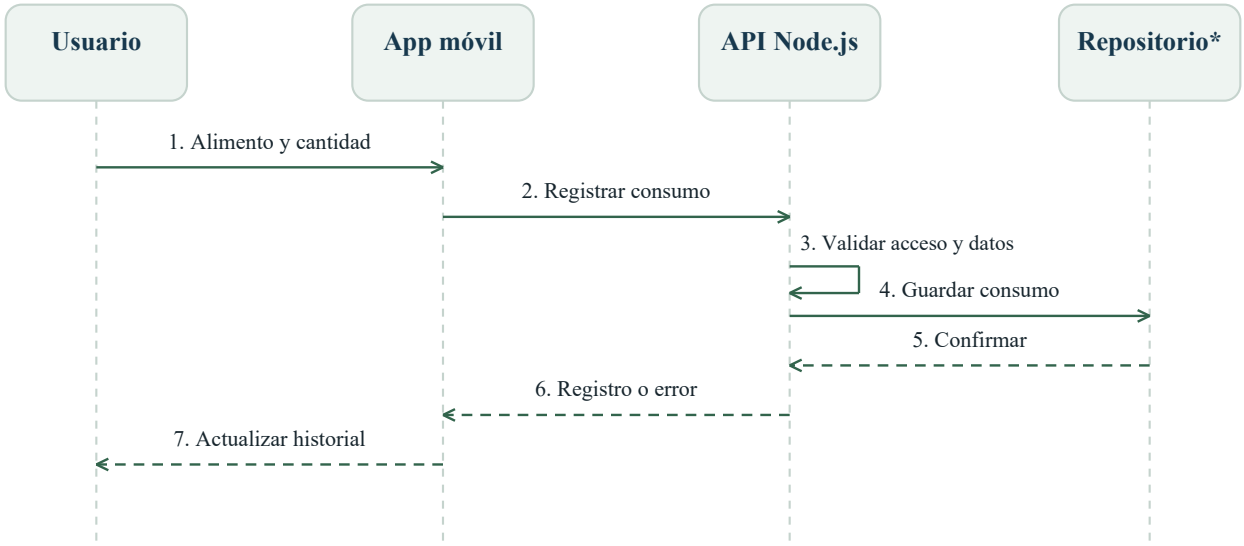


Figura 7. *Registrar un alimento consumido*



*Nota. * Repositorio futuro. Se propone validar sesión, propiedad y contenido en el servidor antes de guardar. Una respuesta de error no debe presentarse como una operación completada.*

En planificación, el guardado de cabecera, comidas y detalles debe conservar consistencia. En consumo, el cliente debe prevenir envíos duplicados mientras espera respuesta. El comportamiento ante reintentos y fallos de red debe definirse y probarse.

Funciones previstas por pantalla y contrato de API

Pantalla prevista	Funciones y límites
Registro y acceso	Crear cuenta, iniciar sesión, presentar errores y recuperar el acceso a la pantalla principal.
Inicio	Entrada a las operaciones del usuario. El resumen visual o los indicadores concretos no están especificados.
Perfil y objetivo	Administrar datos personales, seleccionar el objetivo y cerrar sesión.
Planificador	Consultar y organizar comidas por día o semana; acceder a edición o eliminación.
Registro de consumo	Seleccionar alimento, ingresar cantidad, unidad y fecha, y confirmar lo consumido.
Historial	Consultar consumos propios. Ubicación y filtros exactos deben definirse en la navegación.
Recetas favoritas	Guardar y consultar recetas personales; catálogo y fuente de recetas por precisar.

Nota. Son responsabilidades de interfaz derivadas de los requisitos y del diagrama de desarrollo. No representan capturas ni pantallas verificadas de una aplicación ejecutable.

Contrato conceptual de comunicación propuesto

Recurso lógico	Operaciones a especificar
Cuentas y sesiones	Crear cuenta, iniciar sesión, cerrar o invalidar sesión según el mecanismo elegido.
Perfil	Consultar y actualizar datos propios, incluido el objetivo.
Planes y comidas	Crear, consultar, editar y eliminar recursos con comprobación de propiedad.
Consumos e historial	Registrar y consultar consumos; concretar edición y eliminación.
Alimentos y recetas	Consultar alimentos y gestionar recetas favoritas propias.

Se propone intercambiar JSON y definir métodos HTTP, estados, validación, paginación y errores antes de implementar el cliente. Los avances no establecen endpoints exactos, códigos de error ni un contrato publicado; no se incluyen rutas ficticias como si estuvieran disponibles.

Seguridad, privacidad y límites operativos

RNF-08 exige manejar de forma segura la información de usuario durante la sesión. Los avances identifican autenticación y middleware, pero no demuestran controles implementados. Las medidas siguientes son propuestas para concretar ese requisito.

Área	Control de diseño y evidencia esperada
Credenciales	Guardar hashes de contraseña en el servidor y evitar contraseñas o credenciales de sesión en registros de diagnóstico. Verificar almacenamiento y respuestas.
Sesión	Definir emisión, vigencia y revocación de la credencial. Limpiar estado al cerrar sesión y tratar la expiración en todas las operaciones protegidas.
Almacenamiento móvil	La vista física usa AsyncStorage para datos temporales. El almacenamiento de secretos requiere una decisión específica; no se presume seguro por usar esa biblioteca.
Transporte	Usar HTTPS en entornos compartidos o publicados y separar la configuración de desarrollo de la de operación.
Autorización	Comprobar en la API la propiedad de cada perfil, plan, comida, consumo y receta. Un identificador enviado por el cliente no debe conceder acceso por sí solo.
Datos personales	Recopilar solo los campos necesarios y definir finalidad, edición, eliminación y retención. Peso, estatura y objetivo no deben aparecer en mensajes públicos ni trazas innecesarias.
Validación y errores	Validar formato, tamaño y rangos en servidor. Mostrar avisos útiles sin revelar detalles internos de infraestructura.
Persistencia y recuperación	Cuando se implemente la base, acordar respaldos, recuperación y borrado coherente. Probar restauración antes de prometer disponibilidad de datos.

Límites funcionales

El objetivo del proyecto es organizar alimentación personal. Los avances no especifican diagnósticos, prescripción clínica ni recomendaciones nutricionales calculadas. El sistema debe presentar con claridad lo que el usuario registra y evitar convertir un objetivo de perfil en una recomendación automática sin un requerimiento y una validación propios.

Verificación: evidencia y plan de pruebas

La revisión confirma la existencia de requisitos y diagramas en los PDF. No se ejecutaron pruebas de la aplicación porque no se entregaron código fuente ni un entorno ejecutable. La siguiente matriz propone comprobaciones y conserva todos los resultados como pendientes.

Prueba	Cobertura y resultado esperado	Estado
P-01	RF-01: crear cuenta válida y rechazar correo repetido o datos incompletos.	Pendiente
P-02	RF-02 y RNF-08: acceder con credenciales válidas, rechazar inválidas, cerrar sesión y bloquear acceso posterior.	Pendiente
P-03	RF-03 y RF-04: guardar perfil y objetivo; rechazar cambios sobre otro usuario.	Pendiente
P-04	RF-05: crear plan diario y semanal; comprobar fechas, comidas y consistencia al fallar el guardado.	Pendiente
P-05	RF-06: registrar consumo válido y rechazar cantidad no positiva o unidad inválida.	Pendiente
P-06	RF-07: consultar historial propio, estado vacío y separación entre dos cuentas.	Pendiente
P-07	RF-08: guardar y recuperar recetas favoritas; resolver guardado repetido.	Pendiente
P-08	RF-09: editar y eliminar registros propios; rechazar ajenos y comprobar dependencias.	Pendiente
P-09	RF-10 y RNF-04: respuestas, errores de red, tiempo de espera y reintentos sin duplicados.	Pendiente
P-10	RNF-01 y RNF-02: tareas críticas en Android e iOS y observación de facilidad de uso.	Pendiente
P-11	RNF-03: comprobar el límite de 3 segundos con carga, red y operaciones previamente acordadas.	Pendiente
P-12	RNF-05, RNF-06 y RNF-07: revisar tipado, entorno Node.js y separación modular en el repositorio.	Pendiente

Registro mínimo de resultados

Cada ejecución debe conservar versión del sistema, datos de entrada, pasos, resultado esperado, resultado observado y evidencia. Para rendimiento se deben añadir dispositivo, red, volumen y tiempos medidos. Los resultados del proyecto de facturación usado como plantilla no se trasladan a Meal Planner.

Brechas y plan de trabajo recomendado

La prioridad inicial es transformar el análisis existente en decisiones y evidencias que puedan revisarse. La secuencia propuesta conserva la arquitectura centralizada y atiende primero las dependencias de datos y acceso.

Etapa	Trabajo pendiente	Evidencia de cierre
1. Consolidar alcance	Acordar RF-09, origen de recetas y alimentos, unidades, versiones móviles objetivo y escenarios de rendimiento.	Requerimientos y criterios de aceptación aprobados para el proyecto.
2. Definir datos y API	Seleccionar motor de persistencia; precisar tipos, relaciones, restricciones, migraciones y contratos de recursos.	Esquema reproducible y contrato de API coherente con el modelo.
3. Resolver autenticación	Implementar registro, acceso, sesión, cierre, perfil y autorización por propietario.	Pruebas de dos usuarios y de acceso rechazado; revisión de almacenamiento de credenciales.
4. Implementar flujos del dominio	Conectar planificación, comidas, consumo, historial y recetas con persistencia. Resolver edición, eliminación y reintentos.	Demostración reproducible y pruebas P-04 a P-09.
5. Validar calidad	Evaluar Android e iOS, rendimiento, usabilidad, seguridad y manejo de errores.	Resultados con condiciones de ejecución y correcciones verificadas.
6. Preparar entrega	Documentar configuración real, arranque, versiones, respaldo y límites de operación. Definir publicación si entra en el alcance.	Guía ejecutada desde un entorno limpio y documentación alineada con la versión entregada.

Mejoras de documentación incorporadas

Este informe unifica los avances, conserva los identificadores de requisitos, distingue planificación de consumo, aclara que la API pertenece al sistema y trata la base de datos como propuesta futura. También añade criterios de aceptación y trazabilidad. Son mejoras del análisis documental; no cambios ejecutados sobre el producto.

Condición para revisar la arquitectura

No se requiere adoptar microservicios para completar el alcance descrito. Si más adelante se demuestra una necesidad de operación independiente, deberá compararse ese cambio con sus costos de despliegue, comunicación, consistencia y mantenimiento.

Guía de operación prevista y preparación técnica

Preparación de un entorno verificable

Los avances no incluyen versiones de dependencias, manifiestos de paquetes, variables de entorno ni comandos de instalación. Por ello, esta guía define el procedimiento que deberá concretarse con el repositorio y evita presentar instrucciones de arranque sin verificar.

1. Obtener el repositorio y revisar sus instrucciones y manifiestos para identificar versiones de Node.js, dependencias y gestor de paquetes.

2. Preparar el cliente React Native con Expo y un ambiente Android o iOS conforme a la configuración real del proyecto.

3. Configurar la dirección de la API accesible desde el dispositivo o emulador; distinguir los entornos de desarrollo y operación.

4. Preparar la persistencia elegida y aplicar su esquema o migraciones cuando esa fase esté implementada.

5. Iniciar backend y cliente con los comandos documentados en el repositorio; comprobar conectividad y manejo de errores.

6. Ejecutar el plan de pruebas, registrar resultados y actualizar esta guía con pasos reproducibles.

Flujo básico de uso esperado

1. Registrar una cuenta e iniciar sesión.

2. Completar el perfil y seleccionar el objetivo alimenticio.

3. Abrir el planificador, elegir un día o semana y organizar las comidas.

4. Registrar por separado los alimentos efectivamente consumidos.

5. Consultar el historial y corregir registros propios cuando corresponda.

6. Guardar y consultar recetas favoritas.

7. Cerrar sesión al terminar o al utilizar un dispositivo compartido.

Comportamiento previsto ante incidencias

Un fallo de red debe conservar la claridad del estado de la operación y permitir un reintento controlado. La sesión vencida debe conducir al acceso sin exponer los datos previos. Mientras no se defina sincronización, no se promete guardar operaciones sin conexión. La recuperación de datos dependerá de la base y de la política de respaldos que se implemente.

Conclusiones

Meal Planner cuenta con una base documental que define funciones, tecnologías y relaciones del dominio. La integración permite presentar un proyecto coherente: una aplicación móvil con React Native, Expo y TypeScript que utiliza una API REST centralizada en Node.js. Los módulos separan responsabilidades dentro del backend; los avances no describen microservicios independientes.

La principal limitación es la distancia entre diseño y evidencia de ejecución. El modelo de datos y las secuencias con repositorio son propuestas para una fase de persistencia, mientras que la vista física indica que todavía no existe una base en uso. Los requisitos quedan documentados, pero su implementación, seguridad, rendimiento y compatibilidad no pueden darse por comprobados con los PDF disponibles.

El siguiente paso consiste en cerrar decisiones de alcance, datos y contratos; implementar los flujos con autorización y persistencia; y registrar pruebas reproducibles. Esa evidencia permitirá actualizar la documentación para describir con precisión lo que la versión entregada realiza.